

StrokeTutor

Master Product + Web Studio Plan

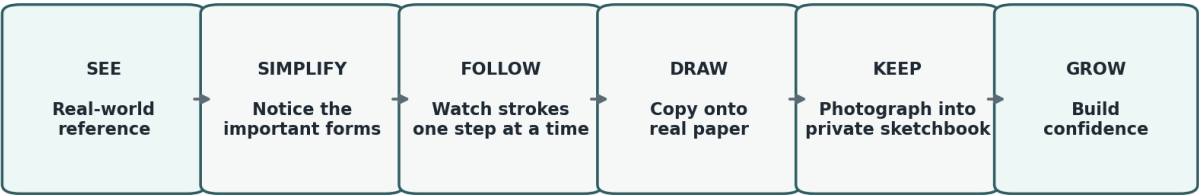
North Star

Help an adult beginner look at a real-world subject and feel: “I know how to start drawing that.”

Web-first plan for building the private StrokeTutor Studio, AI-assisted lesson production pipeline, shared content contract, and the eventual iOS learner experience.

Version 1.0 | September 2026 | Master planning source for the main coding/planning agent

Learner Journey



The app succeeds when the learner stops seeing a blank page as a test and starts seeing a subject as a manageable sequence of choices.

This document consolidates the product decisions, technical PoC baseline, authoring workflow, AI strategy, iOS experience, roadmap, quality gates, and deferred ideas discussed during the current planning session.

How to Use This Master Plan

This PDF is intended to be the durable source of direction for the project. A planning or coding agent should read it before proposing architecture changes, feature work, schema changes, or content-generation workflows.

Three kinds of information are separated intentionally

EXISTING describes facts already present in the PoC/repository. CONFIRMED describes product choices made during planning. RECOMMENDED describes the current implementation direction and may be refined if implementation evidence suggests a better approach.

Priority order when decisions conflict

1. Protect the learner outcome and product principles.
2. Preserve the shared web/iOS content contract and conformance discipline.
3. Keep the user-facing product simple; put complexity into the Studio and tooling.
4. Prefer a small, excellent vertical slice over broad feature coverage.
5. Use real implementation feedback to refine recommendations, but document intentional departures from this plan.

Contents

- Part I - Product North Star and learning model
- Part II - Existing PoC and shared contract
- Part III - StrokeTutor Studio (build first)
- Part IV - AI lesson-generation system
- Part V - Repository, data, and publishing architecture
- Part VI - Eventual iOS learner application
- Part VII - Delivery roadmap, testing, risks, and decision register
- Appendices - examples, first path, and agent handoff checklist

Executive Summary

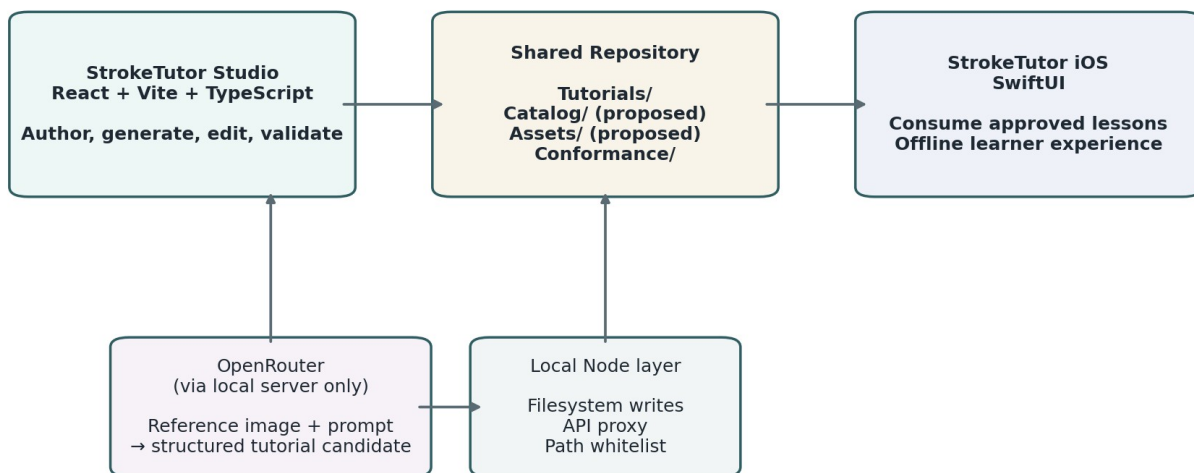
StrokeTutor is not a digital drawing canvas. It is a guided physical-sketching learning system for adults, especially people in midlife and later who always wanted to draw but never found an approachable way to begin. The learner uses a real pen and real paper; the phone acts as a calm instructor.

The core interaction is deliberately simple: the app animates a meaningful drawing step, stops, and waits. The learner copies that step onto paper and explicitly taps “I drew it” before anything continues. The application never advances on its own.

The product should build confidence through repeated successful drawings from real-world subjects. Paths such as Houses, Trees, Landscapes, Flowers, Streets, and Architecture progress from simple forms toward more expressive and complex drawings. Learners may switch paths freely, but lessons inside a path are sequentially locked so difficulty rises gradually and the sketchbook becomes a visible record of progress.

The project should be built web-first. StrokeTutor Studio is the internal authoring product used to upload a real reference photo, describe the learning goal, ask an OpenRouter-powered AI pipeline to generate a simplified pen-and-ink tutorial, review and restructure the strokes/steps, validate the result, preview the exact learner playback, and save approved content directly into the same repository consumed by iOS.

Recommended Web-First System Architecture



Studio creates content; shared files remain the contract; iOS stays a focused consumer player.

First major milestone

Prove that the Studio can repeatedly turn a reference photograph into a high-quality, under-five-minute, human-drawable lesson with modest manual correction. Do this with one short Houses vertical slice before expanding the consumer app or content library.

Part I - Product North Star and Learning Model

1. The problem StrokeTutor solves

The target learner can see something beautiful in the real world but does not know how to translate it onto a blank page. They may have tried videos, books, or drawing apps and still feel unsure where to begin, which details matter, what to ignore, and what line should come next.

The pain is not simply “I cannot draw.” It is the combination of blank-page anxiety, unclear starting points, overcomplicated references, and tutorials that do not feel designed for an adult beginner.

Product transformation

Blank-page anxiety -> guided success -> repeated pattern recognition -> confidence -> focused enjoyment.

2. Primary target audience

Audience characteristic	Product implication
Adults first, especially ~40+ beginners	Mature tone, readable typography, calm pace, generous tap targets, no childish visual language.
No formal art training required	Teach through doing and observation rather than terminology-heavy lessons.
Interested in real-world subjects	Prioritize houses, trees, architecture, landscapes, plants, streets, nature, and everyday places.
Uses a physical sketchbook	Phone is the guide; the real paper drawing is the creative outcome.
May have limited time	Most lessons target roughly 3-5 minutes.
Wants confidence, not judgment	No accuracy scores, AI grades, public likes, or competitive leaderboards in the core experience.

3. Desired long-term learner outcome

After months of use, a learner should be able to sit at a café, in a park, while traveling, or at home, see a house/tree/scene, recognize the important forms, choose where to begin, and start sketching without immediate frustration. The ideal mental state is focused, locked-in, enjoyable, and curious. Coloring with pencils or markers may be an optional finishing activity, but the drawing skill remains primary.

4. Learning Principles

Watch -> Draw -> Confirm -> Continue

This is the core interaction and should remain sacred. A step plays; the app stops; the learner draws on paper; the learner decides when to continue. No timer decides readiness.

Guided success before forced independence

Early use should maximize successful, attractive outcomes. A learner who follows a tutorial and produces a drawing better than they expected accumulates evidence that “I can draw.” Do not immediately force independent redraws that may create frustration. Independence should emerge from pattern recognition and confidence over time.

Teach simplification, not tracing

Each lesson should connect a real reference to the final sketch. The learner gradually learns that an apparently complex subject can be reduced into a manageable set of forms, proportions, landmarks, and meaningful lines.

One teaching step is not always one SVG stroke

A lesson step represents a meaningful action. It may contain one stroke or several related strokes. For example, two window outlines may belong to a single “Add the windows” step. The existing simple-house sample already demonstrates this pattern with two separate strokes inside one windows step.

Quality over volume

A path does not need dozens of repetitive subjects. Roughly 8-12 carefully chosen variations may be enough to expose the recurring patterns of a category while avoiding boredom. Content should be sequenced for transfer, not generated merely to increase a library count.

5. Drawing Language and Content Style

StrokeTutor should develop a recognizable pen-and-ink visual language. It should feel achievable by a capable human hand, not mathematically perfect and not deliberately messy.

Prefer	Avoid
Clean pen-and-ink lines with slight natural imperfection	Perfectly geometric/vector-looking construction
Meaningful continuous pen gestures	Hundreds of tiny disconnected vector segments
Recognizable real-world structure	Cartoon/anime/cute-kids styling
Selective detail that improves recognition or depth	Texture everywhere or excessive crosshatching
Few purposeful pen lifts where natural	Artificially forcing every object into one impossible stroke
Drawings finishable in about five minutes	Beautiful but impractical illustrations that take 20+ minutes

Internal content rule

Every stroke should have a purpose: structure, recognition, depth, character, important texture, or clarity.

Reality -> interpretation -> drawing

Before drawing begins, the learner should see what exists in the real world and what they are trying to produce. The final iOS experience should keep the reference available during the tutorial so the learner continually connects reality to artistic simplification. The Studio should use the same final drawing that the iOS tutorial produces - the completed tutorial strokes themselves are the final illustration.

6. Learning Paths and Progression

The primary organizational model is a Path. The learner chooses which type of subject they want to explore. There is no wrong starting path. Progress is remembered independently per path, and the learner may switch paths at any time.

Candidate initial path set

Path	Possible progression emphasis
Houses	Front-facing forms -> roof/chimney/porch -> side wall -> shallow perspective -> small scene
Trees	Basic trunk/canopy -> pine/bare/wide/tall -> species character -> overlap -> tree in scene
Flowers	Simple bloom -> stem/leaves -> layered petals -> grouped flowers -> plant composition
Mountains	Silhouette -> layered ranges -> foreground elements -> simple mountain scene
Landscapes	Horizon/ground -> major masses -> depth -> small composed scenes
Streets & Places	Bench/lamp/storefront -> doorway -> street corner -> simple urban scene
Plants & Leaves	Leaf shapes -> branching -> potted plant -> larger foliage groups
Water & Coast	Shoreline -> reflections/rocks -> boats/docks -> coastal scene
Everyday Objects	Simple familiar objects with shape/proportion transfer
Architecture Details	Windows/doors/arches/roofs/facades as reusable visual vocabulary

This is a candidate set, not a locked launch list. The content production workflow should be proven before the final launch catalog is frozen.

Locking model

- Inside a path, lessons unlock sequentially so difficulty and confidence rise together.
- Across paths, users can switch freely. A learner may be on Trees 5 and Houses 2 without losing progress.
- A future premium entitlement may optionally unlock all lessons, but monetization should not dictate the learning architecture.
- Future lessons can be appended to an existing path without invalidating prior completion.

7. What a Good Lesson Feels Like

A lesson should produce a finished drawing that an adult beginner would be happy to place in a physical sketchbook. The lesson should be approachable enough to start immediately but visually satisfying enough to create pride.

Lesson anatomy

1. Reference: show the real-world subject.
2. Preview: show the finished StrokeTutor pen drawing.
3. Orientation: optionally explain one concise learning idea.
4. Guided drawing: animate a meaningful step and stop.
5. Learner action: copy onto real paper.
6. Confirmation: learner taps “I drew it.”
7. Repeat until complete.
8. Completion: invite the learner to photograph the page into the private sketchbook.

Time target

Design for approximately 3-5 minutes of guided drawing for most early lessons. The exact number of strokes is secondary to natural pacing and drawability.

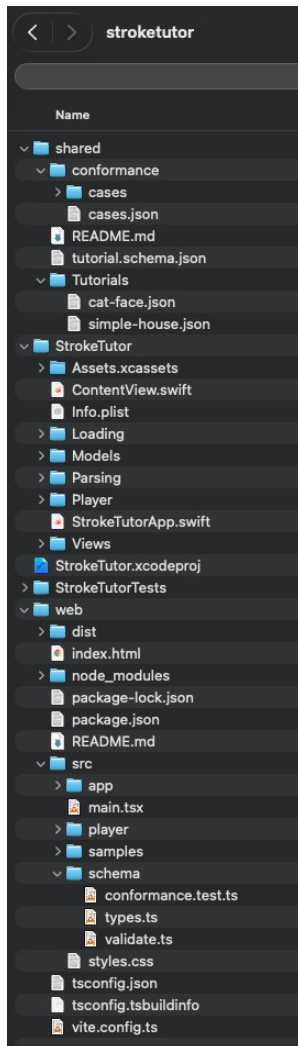
Private sketchbook

The finished physical drawing can be photographed and stored locally. The gallery is not a social feed. It is the learner’s personal record of progress. Valuable future moments include comparing the first tree to a later tree, seeing the number of finished pages, or revisiting a completed path.

Part II - Existing PoC and Shared Contract

8. Current repository baseline - EXISTING

The existing repository is already organized as a shared-contract project: root-level shared content and conformance data, a SwiftUI iOS app, iOS tests/Xcode project, and a React/Vite/TypeScript web player.



Current repository tree provided during planning.

```
stroketutor/
  shared/
    tutorial.schema.json
    Tutorials/
      conformance/
        StrokeTutor/
        StrokeTutorTests/
        StrokeTutor.xcodeproj
      web/
```

The important architectural fact is that the web and iOS sides consume the same tutorial files rather than maintaining copies. That should continue.

9. Current tutorial contract - EXISTING

```
Tutorial v1
  schemaVersion
  id
  title
  canvas { width, height }
  optional style { strokeColor, backgroundColor }
  steps[]
    id
    title
    instruction
    optional voiceover
    strokes[]
      d
      duration
      lineWidth
```

The schema is strict on the authoring/web side: unknown properties are rejected. The allowed SVG path subset is absolute M, L, C, Q, and Z. Relative commands and unsupported commands such as arcs are rejected rather than risk different rendering between platforms.

10. Current Playback Contract - EXISTING

```
idle
-> drawing(step)
-> awaitingUser(step)
-> drawing(next step)
-> ...
-> finished
```

Strokes inside a step play sequentially. Completed work remains visible while future work is hidden. The player has replay/back/restart and speed controls. The learner controls when a completed step advances.

Current sample: simple-house

Step	Teaching intent	Stroke grouping
Walls	Main house body	1 closed path
Roof	Roof triangle	1 closed path
Door	Door outline	1 open path
Windows	Two windows together	2 separate strokes in one teaching step
Chimney	Finishing detail	1 path

This sample is important because it proves the existing v1 model already supports the Studio behavior we want: multiple strokes can be grouped into one meaningful teaching step without changing the schema.

11. Conformance philosophy - EXISTING

The shared conformance corpus is more important than the JSON Schema alone because it captures SVG grammar behavior and deliberate differences between web and iOS. Both test suites consume the same cases.

Web authoring side	iOS learner side
Strict: rejects bad duration, line width, unknown keys, malformed colors, and unsupported SVG.	Lenient where necessary so a learner is not stranded mid-lesson; some bad values are clamped or ignored with warnings.
Purpose: catch bad content before shipping.	Purpose: play a shipped lesson safely even if content is imperfect.

Preserve this asymmetry

Studio should remain strict. iOS should remain forgiving where the existing contract intentionally says so. Do not weaken tests simply because an AI generator produces invalid output.

12. What Must Be Preserved While Building

- The shared/tutorial.schema.json v1 playback contract unless a deliberate versioned migration is approved later.
- The same SVG parser semantics in TypeScript and Swift.
- The shared conformance corpus and explicit “why” for any intentional divergence.
- TutorialPlayer as a reusable validated-tutorial component rather than replacing it with a Studio-specific fake player.
- Web validation before content can be treated as approved.
- The idea that shipped content is consumed from a shared source of truth, not copied independently into web and iOS.

Recommended near-term rule

Keep tutorial.schema.json v1 unchanged while the Studio is being proven. Put curriculum, authoring, and asset metadata beside the tutorial rather than inside it. This limits risk to the proven playback contract.

Part III - StrokeTutor Studio (Build This First)

13. Studio mission

StrokeTutor Studio is a private, local, AI-assisted content production application. It is not a consumer website and should not look like a generic SaaS dashboard. The creator should spend most of their time in a lesson workspace, not in analytics.

Studio core loop

Choose Path -> Create Lesson -> Upload Reference -> Describe Learning Goal -> Generate -> Review -> Reorder/Group -> Preview -> Validate -> Approve & Save

14. Recommended top-level information architecture

Area	Purpose
Paths	See the curriculum, lesson order, completion of authoring work, and progression for each subject family.
Lesson Workspace	Primary working surface: reference image, generated drawing, step/stroke structure, instructions, and preview.
Generate	Upload a reference and ask AI to produce a tutorial candidate using path/curriculum context.
Review / Needs Attention	Find validation failures or generated lessons waiting for human approval.
Settings	OpenRouter provider/model settings, prompt configuration, and advanced Studio preferences.

A Dashboard can exist, but it should be lightweight. The Studio is a creative production tool; the Path and Lesson Workspace are more important than reporting widgets.

15. Path Builder

Each path should show the lesson progression visually and make it obvious where a new lesson fits. Drag-and-drop reordering is useful because curriculum sequencing will evolve while testing.

HOUSES

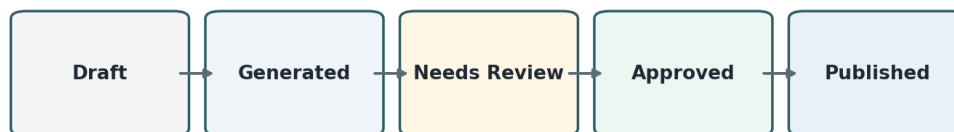
01	Simple House	Approved
02	House + Chimney	Approved
03	Small Cottage	Needs Review
04	House From an Angle	Draft
05	Two-Story House	-

Lesson metadata visible in the path view

- Title and thumbnail / finished-stroke preview.
- Order inside the path.
- Authoring status.
- Estimated learner time.
- Complexity/stage.
- One-line learning objective.
- Warnings such as “too many strokes” or “validation failed” when applicable.

Suggested authoring statuses

Lesson Lifecycle



Approval is human. "Published" initially means the validated lesson package has been saved into the shared repository for the app to consume.

16. New Lesson Flow

A new lesson should require enough context for the generator to make a curriculum-aware decision without making the creator fill out a large form.

Input	Example
Path	Houses
Position	Lesson 4 of the current path
Reference image	Creator uploads a real-world house photograph
Learning goal	Introduce a visible side wall and shallow perspective while staying beginner-friendly
Optional constraints	Under 5 minutes; avoid landscaping; preserve chimney and main windows
Complexity target	Beginner / next step after the previous lesson

Primary action

Generate Tutorial

One primary button should run the full pipeline. Advanced stage-level controls appear after generation so the creator can regenerate only the weak part.

Why a single button matters

The normal authoring experience should be easy enough that creating a lesson feels creative rather than technical. The deeper pipeline should be visible only when the creator needs to debug, compare, or refine the AI output.

17. Lesson Workspace - The Most Important Studio Screen

The workspace should place the real-world reference, generated tutorial drawing, and teaching structure in one place. The creator should be able to judge both artistic simplification and teaching flow without navigating between screens.

Houses / Cottage		Draft	Save
REFERENCE	DRAWING	STEPS	
[photo]	[SVG canvas]	1 Walls	
		2 Roof	
		3 Windows	
		4 Door	
Prompt / Instruction / Stroke Inspector / Advanced			

The final drawing shown in the workspace should be all tutorial strokes rendered complete. Do not maintain a separate final SVG asset in v1. This guarantees that the Studio's final image and the iOS tutorial's final state are identical.

18. Human Editing: Guide the Lesson, Do Not Redraw It

The creator does not want a freehand vector editor. The creator wants to guide AI-generated content so it teaches in a natural order.

Required v1 manipulation

Action	Use case
Select one stroke	Inspect, delete, move, replay, or edit duration/line width.
Multi-select strokes	Group related marks such as two windows.
Group into new step	Turn selected strokes into one meaningful teaching action.
Move to existing step	Correct AI grouping without regenerating the drawing.
Split step	Separate a step that asks the learner to do too much at once.
Merge steps	Combine overly granular actions.
Reorder steps	Change teaching order, e.g., walls before roof.
Reorder strokes inside step	Change left/right or inner sequence while keeping the step.
Edit title/instruction	Make language concise, adult, and observational.
Replay stroke / step / lesson	Judge the actual experience immediately.

Useful Studio-only visualization

In Edit Mode, temporary colors can distinguish steps or selected strokes. In Learner Preview, all temporary authoring colors disappear and the actual tutorial style is rendered.

19. Preview and Debugging

The existing player should remain the authoritative learner preview. After editing, “Preview as Learner” should mount the real validated tutorial in TutorialPlayer rather than a separate approximation.

Keep and improve the existing debug tools

- Coordinate grid and axis labels.
- Live cursor readout in canvas coordinates.
- Stroke inspector with path data, duration, and measured length.
- Hover/select highlighting.
- Optional stroke numbers and direction/start/end indicators.
- Advanced panel rather than permanent visual clutter.

Exact-preview rule

If the Studio preview and the iOS result can diverge visually or behaviorally, authoring becomes untrustworthy. Reuse the validated player and shared files wherever possible.

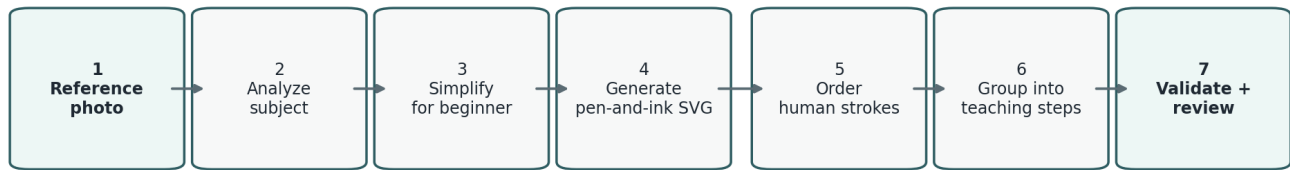
Part IV - AI Lesson Generation System

20. AI strategy

AI should be powerful behind the scenes and quiet in the consumer app. The first version of the Studio uses creator-uploaded reference images. A future AI agent may generate photorealistic source references, but that is explicitly deferred.

OpenRouter is the preferred provider abstraction. The Studio should not hard-code itself to one model; model selection may evolve as image understanding, structured output, and SVG generation quality are tested.

AI Generation Pipeline - One Button on the Surface, Staged Underneath



Each stage can later be regenerated independently so a good drawing is not discarded because grouping or instructions are weak.

21. Recommended staged pipeline

Stage	Responsibility
Reference analysis	Identify silhouette, proportions, important landmarks, complexity, and details that can be ignored.
Simplification plan	Decide what a beginner actually needs to represent for the subject to remain recognizable.
Pen-and-ink SVG generation	Create the final drawable interpretation using the supported path grammar.
Human stroke ordering	Order paths as plausible pen gestures, not arbitrary vector drawing order.
Teaching-step grouping	Group one or more strokes into meaningful learner actions.
Instruction generation	Write concise adult-facing step titles/instructions.
Validation	Schema + SVG grammar + Studio quality checks before human review.

22. AI Output Contract

Do not ask the model for an opaque SVG blob only. The Studio should request structured authoring output with both reasoning metadata for the creator and an exact tutorial candidate for the player.

```
{
  "analysis": {
    "mainForms": [...],
    "importantDetails": [...],
    "detailsRemoved": [...],
    "drawingStrategy": "..."
  },
  "tutorial": {
    "schemaVersion": 1,
    "id": "...",
    "title": "...",
    "canvas": {...},
    "steps": [...]
  }
}
```

Only tutorial is eligible to become the shared playback document. Analysis can remain Studio metadata or generation history.

Non-negotiable SVG constraints

- Absolute M, L, C, Q, Z only.
- No relative path commands.
- No arcs or unsupported commands.
- No zero duration or zero line width.
- No unknown fields in tutorial v1.
- Every step must contain at least one stroke.
- Every path must be a plausible human pen movement, not a dense auto-trace.

23. Making AI Drawings Feel Human

A technically valid SVG can still fail the product if it feels synthetic, over-perfect, or unpleasant to reproduce. Human drawability is a first-class quality dimension.

Generation principles

- Prefer a few confident lines over dense vector tracing.
- Allow subtle irregularity so rectangles, roofs, branches, and curves do not look machine-perfect.
- Preserve recognizable proportions and characteristics while removing decorative noise.
- Minimize unnecessary pen lifts, but do not force an unnatural single-line drawing.
- Avoid tiny paths created only because an image-vectorizer found edges.
- Design for a beginner's hand speed and under-five-minute lesson target.
- Let a stroke correspond to a meaningful physical gesture whenever possible.

Automatic warnings (recommended)

Warning	Why it helps
High stroke count	May indicate over-detailed/vectorized content.
Many extremely short strokes	Likely unnecessary texture or tracing artifacts.
Estimated lesson > 5 minutes	Flags lessons drifting from the core promise.
Unsupported SVG command	Prevents cross-player divergence.
Very large complexity jump from prior lesson	Protects path progression.
Step with too many unrelated strokes	May feel cognitively heavy for a beginner.

These should be warnings, not rigid automatic rejection, except where the existing schema/parser contract already requires rejection.

24. Regeneration and Prompt Management

Generation should not be all-or-nothing. If the drawing is good but grouping is weak, preserve the drawing and regenerate or manually edit only the weak layer.

Control	Purpose
Regenerate drawing	Try a different simplification/visual interpretation.
Regenerate stroke order	Keep the image but improve human drawing sequence.
Regenerate steps	Keep strokes but regroup into better teaching actions.
Regenerate instructions	Keep the lesson structure and improve language.
Compare generations	Retain good prior versions instead of overwriting blindly.

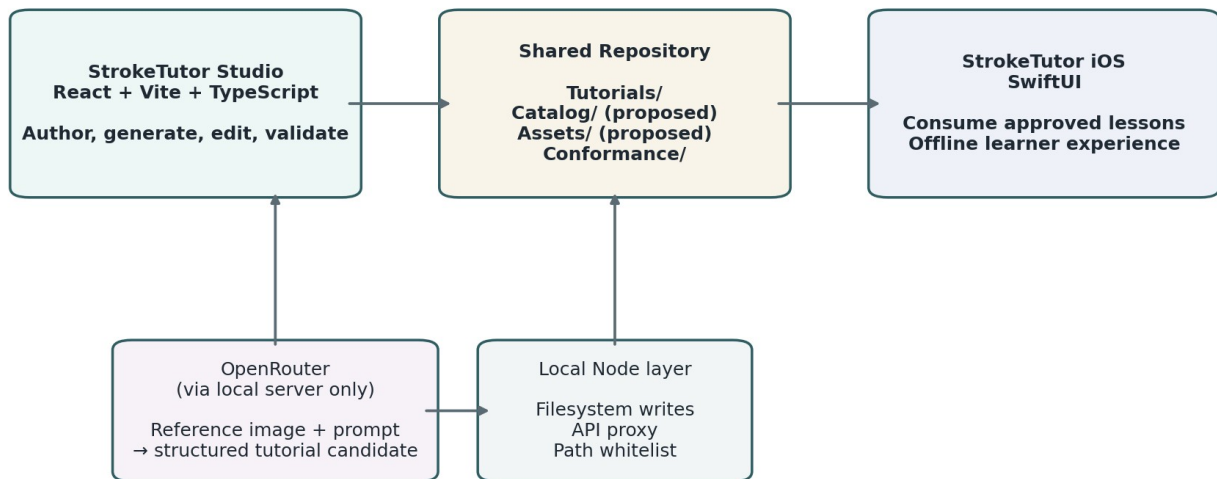
Prompt Lab - later Studio capability

Prompt quality will directly affect content quality. Prompt versions for analysis, simplification, SVG generation, grouping, and instructions should eventually be editable and recorded with generation history. This does not need to block the first working end-to-end generator.

Part V - Repository, Data, and Publishing Architecture

25. Recommended web-first architecture

Recommended Web-First System Architecture



Studio creates content; shared files remain the contract; iOS stays a focused consumer player.

The browser should not call OpenRouter directly and should not be responsible for arbitrary filesystem writes. Add a small local Node-side service within the web package (or equivalent Vite server middleware) that can safely proxy OpenRouter and write only to approved repository paths.

Why a local server layer

- Keeps the OpenRouter API key out of the browser bundle.
- Allows “Save Lesson” to write directly into the Git repository.
- Allows copying reference assets into the shared content tree.
- Creates a single place for filesystem path whitelisting and validation.
- Can run strict validation before writing approved content.

Security boundary

The file API should only write beneath explicitly allowed project directories. Never accept an arbitrary client-supplied filesystem path.

26. Shared Repository Layout - Recommended Evolution

Current PoC content lives in shared/Tutorials and shared/conformance. The recommended near-term evolution adds separate curriculum metadata and reference assets without changing the v1 playback schema.

```
shared/
  tutorial.schema.json          # existing playback contract
  Tutorials/
    simple-house.json          # existing, canonical playable tutorial JSON
    ...
  conformance/                 # existing parity contract
    cases/
      cases.json
  Catalog/                     # proposed Studio/iOS curriculum metadata
    paths.json
    lessons.json
  Assets/                      # proposed shared lesson assets
  References/
    house-cottage-01.jpg
    ...
```

The exact metadata format can evolve, but the separation is important: tutorial JSON answers “how do I play this drawing?”; catalog metadata answers “where does this lesson live in the curriculum and which assets belong to it?”

Example path metadata - conceptual

```
{
  "id": "houses",
  "title": "Houses",
  "lessonIds": [
    "simple-house",
    "house-chimney",
    "small-cottage",
    "angled-house",
    "two-story-house"
  ]
}
```

27. Save / Approve / Publish Behavior

For the first Studio, “publish” can be intentionally simple. There is no need for a content server or production CMS.

1. Creator opens a generated or edited lesson.
2. Studio runs strict schema and SVG validation.
3. Studio runs additional quality warnings.
4. Creator explicitly approves the lesson.
5. Local server writes tutorial JSON into shared/Tutorials/.
6. Local server stores/copies the reference image into the proposed shared asset location.
7. Catalog metadata is updated.
8. Web player reloads the exact saved file as a verification pass.
9. Existing npm tests should remain green; iOS conformance tests remain the cross-platform gate.

The Studio should save repository files; it does not need to perform git commits in v1. Normal source control remains visible as diffs so the developer/agent can review content changes before committing.

Privacy distinction

Consumer iOS app	Private Studio
Learner sketchbook should remain local/private; no account or upload required.	Creator reference photos and prompts are sent to the selected OpenRouter model when generation is requested.
Core learning works offline.	AI generation requires network access; authoring/playback/editing can still work locally once content exists.

28. Studio MVP - Definition of Done

Do not begin with every future Studio idea. The first useful Studio proves the complete production loop.

1. Create and reorder a Path.
2. Create a lesson inside a Path.
3. Upload a reference photograph.
4. Enter a learning goal / creator prompt.
5. Generate or import a schema-valid tutorial candidate.
6. See all strokes and steps over the reference context.
7. Reorder steps and strokes.
8. Group, split, and merge steps without drawing paths manually.
9. Edit instructions, duration, and line width.
10. Preview through the existing learner player.
11. Run strict validation.
12. Approve and save into the shared repository.
13. Reopen the saved lesson from shared/Tutorials and confirm it plays identically.

First Studio success criterion

The creator should be able to turn one reference image into a satisfying lesson without hand-writing JSON or SVG paths, and should be able to correct AI teaching order without regenerating everything.

Part VI - Eventual iOS Learner Application

29. Consumer product principle

The iOS app should remain simple on the surface and deep through content. The learner should feel as though they opened a sketchbook and a patient teacher sat beside them - not as though they opened a feature-heavy productivity application.

Primary areas

Area	Purpose
Onboarding	Reiterate the beginner's pain, show the transformation, explain the five-minute physical-paper method.
Home / Paths	Ask what the learner wants to sketch and show calm progress across subject paths.
Path Detail	Show ordered progression, next available lesson, completed lessons, future locked lessons, and destination examples.
Lesson Preview	Reference photo, final sketch, approximate time, one concise learning objective.
Player	Reference + animated tutorial + instruction + "I drew it" + quiet playback controls.
Completion	Mature celebration and invitation to capture the physical drawing.
Sketchbook	Private local gallery of photographed drawings organized by lesson/path/date.
Settings	Minimal; speed/accessibility/privacy/reset where useful.

30. iOS First-Run Experience

Onboarding should begin with the learner's emotional problem and end with a believable outcome, not with a feature tour.

1. "Have you ever looked at something you wanted to draw..." - show a real subject.
2. "...then looked at the blank page and had no idea where to begin?"
3. Explain that drawing gets easier when you learn what to notice and what to leave out.
4. Show real reference -> simplified interpretation -> finished pen sketch.
5. Explain the method: watch one small step, draw it on paper, continue when ready.
6. Emphasize "five minutes, a pen, and paper."
7. Show a progression from first simple drawing toward a more beautiful later drawing.
8. Ask: "What would you like to learn first?" and let the learner choose a Path.

Tone

Use	Avoid
"Nice work. Your cottage is finished."	"AMAZING!!! +500 XP!!"
Calm, direct, capable adult language	Childlike praise or mascot-driven motivation
Progress through finished drawings	Competitive leaderboards or public comparison
Visible path destination	Pressure-heavy daily streak mechanics

31. Player Experience

The player remains the heart of the product. The real-world reference should remain available during drawing - side-by-side where practical, or as a clear persistent/expandable reference on smaller screens.

Visual priority

1. Real-world reference / connection to reality.
2. Current tutorial drawing canvas.
3. Concise step instruction.
4. Progress such as “Step 4 of 9”.
5. Primary “I drew it” action.
6. Secondary controls: replay, back, speed, restart.

Do not use countdowns. Do not auto-advance. The learner’s hand and attention decide the pace.

Accessibility

- Readable type and generous tap targets for older adults.
- High contrast and simple navigation.
- Dynamic Type where practical.
- Minimize hidden gestures.
- Reduced-motion support for non-drawing UI motion while preserving essential tutorial animation in a sensible form.
- VoiceOver-friendly controls and labels, recognizing that the drawing content itself is visual.
- Consider landscape orientation when it materially improves reference + drawing visibility.

32. Completion and Private Sketchbook

At completion, do not grade the drawing. The application has already structured the task to produce a satisfying result. The final action is to help the learner keep evidence of progress.

Completion message example

Your cottage is finished. Add it to your sketchbook.

Sketchbook principles

- Photograph the physical page after a tutorial.
- Store locally and privately by default.
- Associate the image with lesson, path, and completion date.
- Allow browsing by path/date without turning it into a social feed.
- Future: optional separate colored version of the same drawing.
- Future: reflective progress moments such as “your first tree vs. your latest tree.”

Do not build yet

- AI accuracy scoring of learner drawings.
- Public profiles, followers, likes, comments, or feeds.
- Competitive leaderboards.
- Mandatory accounts/cloud sync.
- Consumer-facing AI “draw anything” generation.
- Complex digital drawing tools or brush systems.

Part VII - Delivery Roadmap

33. Recommended build sequence from beginning to end

Phase	Goal	Exit condition
0 - Protect PoC	Run and understand existing web/iOS tests and player behavior.	Baseline tests/build documented and green.
1 - Studio shell	Turn web app into an authoring shell without replacing the player.	Path + lesson navigation works with existing sample.
2 - Lesson editor	Make existing JSON editable through step/stroke operations.	simple-house can be reordered/grouped/split and re-previewed without manual JSON.
3 - Repo writer	Add local Node API for safe filesystem writes.	Edited tutorial can validate, save to shared/Tutorials, reopen, and match preview.
4 - OpenRouter	Secure provider integration and structured output.	Uploaded reference + prompt produces a candidate tutorial and handles failures cleanly.
5 - AI refinement	Improve human-drawable SVG and stage-level regeneration.	Most candidate lessons need only modest structural correction.
6 - Houses vertical slice	Author ~5 strong lessons end to end.	Path demonstrates meaningful progression and all lessons pass quality gates.
7 - iOS product shell	Build onboarding/home/path/preview around approved content.	Learner can choose path and complete saved Studio lesson.
8 - Sketchbook	Add local photo capture/gallery/progress.	Finished physical drawing can be kept privately and linked to completion.
9 - Content expansion	Add more paths only after workflow is repeatable.	Creator can scale content without engineering each lesson.
10 - Monetization / distribution	Only after core experience and retention are understood.	Product has evidence of value before paywall complexity.

34. Phase 0-3 Detailed Checklist: Build the Studio Before AI

Phase 0 - Baseline

- Run web npm test and npm build.
- Run iOS conformance tests in the expected simulator environment.
- Confirm simple-house and cat-face play on both sides.
- Document the current deliberate web/iOS divergences.
- Do not modify schema/parser behavior until the editor can preserve existing tutorials.

Phase 1 - Studio shell

- Add Paths and Lesson Workspace around the existing player.
- Load current shared/Tutorials as the content source.
- Introduce proposed catalog metadata outside tutorial.schema.json.
- Make simple-house appear inside a sample Houses path.

Phase 2 - Editing

- Visual stroke selection.
- Step/stroke reorder.
- Group, split, merge, move.
- Instruction/title edits.
- Duration/lineWidth edits.
- Replay selected stroke/step.
- Learner Preview using existing TutorialPlayer.

Phase 3 - Repository writer

- Local server endpoint with strict path whitelist.
- Write tutorial JSON safely.
- Write reference asset safely.
- Update catalog metadata.
- Validate before approval/save.
- Reload exact saved file and show save result.

35. Phase 4-6 Detailed Checklist: Add AI, Then Prove Houses

Phase 4 - OpenRouter integration

- API key remains server-side (environment configuration).
- Provider/model selection is abstracted enough to change models later.
- Reference image + path context + creator prompt are sent only on explicit Generate.
- Responses are parsed as structured output and validated before entering the editor.
- Failure states preserve creator input and never overwrite an existing approved lesson.

Phase 5 - Generation quality

- Enforce supported SVG commands in the prompt and post-validation.
- Prompt for plausible human pen gestures rather than generic vector trace.
- Add stage-level regeneration.
- Add warnings for high stroke count / tiny strokes / excessive duration.
- Keep generation history once useful enough to justify it.

Phase 6 - Houses vertical slice

Lesson	Learning progression idea
1. Simple House	Main front shape + roof; immediate success.
2. House With Chimney	Reuse base form; add one recognizable architectural feature.
3. Small Cottage	Add character with selective windows/door/roof details.
4. House From an Angle	Introduce visible side wall and shallow perspective.
5. Two-Story House	Increase proportion/structure while staying within five minutes.

These lesson names are a test slice, not the final complete Houses path. The point is to prove the authoring workflow and progression logic with enough variety to expose problems.

36. Testing and Quality Gates

Existing gates to preserve

- Web schema validation for every loaded/edited tutorial.
- Web SVG parser validation using the supported grammar.
- Shared conformance corpus run by web and iOS tests.
- Web build/type-check.
- iOS player should continue to tolerate its documented deliberate divergences rather than becoming stricter by accident.

New Studio tests recommended

Test area	Examples
Editor transformations	Group/split/merge/reorder round-trip preserves every stroke exactly once.
Catalog	Reorder lessons and save/reload stable IDs/order.
File API	Reject path traversal; only write whitelisted shared locations; safe atomic writes.
OpenRouter parsing	Malformed/missing fields never enter Approved state.
Generation validation	Unsupported commands and bad numbers are surfaced with exact field/path.
Preview parity	Saved tutorial reloaded from shared/Tutorials produces same stroke/step order.
Regression fixtures	Keep representative approved AI output as fixtures so editor/parser changes are testable without calling models.

Human content quality checklist before approval

- Would an adult beginner be proud of the finished drawing?
- Does it look human rather than machine-perfect?
- Can it reasonably be completed in ~5 minutes?
- Does each step feel like one understandable action?
- Is the stroke order natural?
- Are there unnecessary details or tiny marks?
- Does the reference-to-sketch simplification make sense?
- Is the difficulty appropriate relative to the previous lesson?

37. Risk Register

Risk	Impact	Mitigation
AI produces visually good but undrawable SVG	High	Evaluate path structure, not screenshot appearance; human stroke editor; short-stroke warnings.
AI output becomes overly geometric	High	Prompt for subtle hand character; reject auto-trace style; curate approved examples.
Schema drift between Studio and iOS	High	Keep v1 stable; shared files + conformance; version future changes deliberately.
AI grouping feels unnatural	Medium	Human group/split/merge/reorder is a core Studio feature; regeneration is stage-specific.
Too much repetitive content causes boredom	Medium	Cap early paths around a purposeful set; each lesson should add variation/skill transfer.
Reference-image rights become unclear	Medium	Store source/license metadata; v1 uses creator-uploaded licensed/free/AI assets.
OpenRouter key leaks to client	High	Server-side proxy only; environment secret; no client key storage.
Local file API writes outside repo	High	Whitelisted paths + safe IDs + no arbitrary filesystem path from browser.
iOS layout becomes crowded with reference + tutorial	Medium	Prototype responsive reference patterns; keep player hierarchy focused.
Monetization distorts learning sequence	Medium	Build learning model first; keep entitlement logic separate from curriculum metadata.
Scope expands before core quality is proven	High	Use Houses vertical slice as gate before iOS shell/content expansion.

38. Decision Register - Confirmed During Planning

Decision	Current direction
Primary audience	Adults first, especially middle-aged/older beginners; not a kids-first app.
Learning goal	Confidence drawing real-world subjects; remove frustration and teach how to begin.
Medium/style	Pen and ink; relatively clean, not excessively loose; meaningful continuous strokes.
Session length	Most lessons should take no more than about five minutes.
Curriculum model	Multiple subject Paths; sequential progression within a Path; free switching between Paths.
Path size	Start small; roughly enough variation (often around ~10) to cover recurring patterns without boredom.
Reference behavior	Show the real-world reference at the beginning and keep it available during drawing.
Completion	Photograph the physical drawing and keep it in a private gallery/sketchbook.
Consumer AI	Not needed initially.
Studio AI	All tutorials intended to be AI-generated then human-reviewed/refined.
AI provider	OpenRouter preferred.
Reference sourcing v1	Creator uploads images; future search/generative reference workflow deferred.
Studio editing	No manual path drawing; creator changes ordering/grouping/splitting/step structure and instructions.
Generate UX	One main Generate Tutorial action; advanced stage controls underneath.
Final drawing	Same completed tutorial strokes in Studio and iOS; no separate final art required for v1.
Repository	Web + iOS remain in same GitHub repository with shared content contract.
Privacy	Consumer sketchbook local/private; no account/backend required for core use.

39. Recommended Decisions Not Yet Irreversible

The following are strong implementation recommendations from the planning discussion, but the main agent may refine them if a proof-of-concept reveals a simpler or safer approach. Any departure should be explicit and justified.

- Keep tutorial.schema.json v1 unchanged while building Studio v1.
- Add separate shared Catalog/ metadata instead of putting path/order/status into playback JSON.
- Add shared Assets/References for the source photo associated with a lesson.
- Use a small local Node service / Vite middleware for OpenRouter and filesystem writes.
- Define “Published” initially as validated content saved into the shared repository, not uploaded to a remote CMS.
- Do not have Studio perform git commits automatically in v1.
- Derive the final sketch from all completed tutorial strokes rather than a separate final SVG asset.

Future decisions intentionally deferred

- Exact launch set of 10 paths and number of lessons per path.
- Which OpenRouter model(s) provide the best reference understanding vs. SVG output.
- Whether premium users can unlock all lessons immediately and what monetization model wins.
- Remote downloadable content/update architecture.
- Cloud sync/accounts for sketchbook, if ever desired.
- Built-in royalty-free image search.
- AI agent that generates real-world reference photos.
- Optional color-reference assets and how they appear in the sketchbook.
- Voice/audio support beyond the existing placeholder.
- How/when to introduce more explicit independent drawing challenges.

40. What the Main Agent Should Do First

Do not start by rewriting the architecture

The current PoC already contains the hardest contract to get right: one tutorial format, two players, and shared conformance. Build the Studio around it first.

1. Read shared/README.md, shared/tutorial.schema.json, shared/conformance/cases.json, web/README.md, and the current simple-house tutorial.
2. Run the current web and iOS tests/builds and record the baseline.
3. Inspect TutorialPlayer, usePlayback, svgPath.ts, PlayerViewModel, SVGPathParser, and TutorialLoader before changing behavior.
4. Write an incremental implementation plan for Phase 1-3 only: Studio shell, editor operations, safe repository writer.
5. Use simple-house as the first editable tutorial and prove grouping/splitting/reordering with no AI involved.
6. Only after the authoring editor and save/preview loop is reliable, integrate OpenRouter.
7. Use five Houses lessons as the first end-to-end generated curriculum slice.
8. Do not expand iOS navigation/product features until the Studio can produce reliable approved content.

Definition of “ready to move beyond the Studio proof”

A reference photo can be uploaded; the creator can describe a learning objective; AI generates a valid candidate; the creator can correct the lesson structure without drawing SVG manually; the Studio previews the exact player; approval saves to the shared repo; tests remain green; and the resulting lesson feels natural and achievable by an adult beginner.

Appendix A - Example Houses Path: Curriculum Thinking

A strong path is not merely a list of harder pictures. Each lesson reuses prior visual vocabulary while adding one manageable idea. This example is intentionally illustrative rather than final.

#	Drawing	Primary new idea	Reuses
1	Simple House	Main body + roof relationship	None
2	House With Chimney	Add one characteristic detail	Body + roof
3	Small Cottage	Selective character/detail	Body + roof + windows/door
4	House From an Angle	Visible side plane / shallow perspective	Prior house vocabulary
5	Two-Story House	Vertical proportion + repeated windows	House structure
6	Barn	Different silhouette and large door	Roof/body relationships
7	Farmhouse	More complex roof/window organization	Multiple prior patterns
8	House + Tree	Combine subject families without full scene complexity	House + basic tree
9	Cottage With Foreground	Layering/depth cues	House + environment
10	Small Street Scene	Apply patterns to a composed real-world place	Most prior skills

Path graduation feeling

After a limited set of varied houses, the learner should not think “I memorized ten drawings.” They should think “I understand how houses are put together enough to attempt a new one.”

Appendix B - Example Studio Workflow for One Lesson

1. Creator opens Houses and chooses “New Lesson.”
2. Creator positions it after the current cottage lesson.
3. Creator uploads a licensed/free/AI-generated real-world house reference.
4. Creator writes: “Introduce a visible side wall and shallow perspective. Keep the roof simple, preserve the front door and two major windows, ignore landscaping. Under five minutes.”
5. Creator clicks Generate Tutorial.
6. Studio sends reference + structured context to OpenRouter through the local server.
7. Studio receives analysis + tutorial candidate and validates it.
8. Lesson Workspace opens with reference, completed-strokes preview, and generated steps.
9. Creator notices roof is taught before walls; drags Walls above Roof.
10. Creator selects both window strokes and groups them into one “Add the windows” step.
11. Creator deletes two tiny decorative strokes.
12. Creator edits the instruction to be calm and adult-facing.
13. Creator selects Preview as Learner and completes the tutorial flow.
14. Studio shows no blocking validation errors and one non-blocking duration warning if relevant.
15. Creator clicks Approve & Save.
16. Local server writes shared/Tutorials/<id>.json, saves reference metadata/assets, updates catalog, and reloads the exact saved tutorial for verification.
17. The repository now contains an approved lesson ready for the eventual iOS Path UI.

Appendix C - Source Baseline Reviewed for This Plan

The plan is grounded in the current PoC materials supplied during planning. These existing artifacts define what is already implemented and should not be silently replaced.

Source	What it establishes
shared/tutorial.schema.json	Strict v1 tutorial fields, step/stroke structure, colors, positive duration/lineWidth, and absolute M/L/C/Q/Z path subset.
shared/README.md	Shared source-of-truth layout, why conformance is the real cross-player contract, deliberate strict-web/lenient-iOS differences, and test commands.
shared/conformance/cases.json	Accepted/rejected cases, parser edge cases, and deliberate platform divergences.
shared/Tutorials/simple-house.json	Concrete example of walls/roof/door/windows/chimney steps and grouping two window strokes into one step.
web/README.md	Existing React/Vite player architecture, loading flows, validation behavior, playback state machine, debug panel, and separation of player from app UI.
Repository screenshot	Current root layout showing shared, StrokeTutor, tests/project, and web in one repository.

Closing principle

Build the content engine first

StrokeTutor becomes compelling when the team can reliably create many carefully sequenced, human-drawable lessons without hand-authoring SVG. Prove that engine in the Studio; then wrap it in the calmest possible learner experience.

Master Plan Summary - One Page

Question	Answer
Who is it for?	Adult beginners, especially people returning to drawing later in life.
What do they do?	Look at a real subject, watch a short stroke/step, copy it onto paper, and continue when ready.
What is learned?	How to simplify real objects, recognize visual patterns, and start drawing with confidence.
How long?	Mostly 3-5 minute lessons.
How is content organized?	Sequential lessons inside switchable subject Paths.
What gets built first?	Private web Studio for AI generation, editing, validation, preview, and repository publishing.
How is AI used?	OpenRouter behind the Studio; uploaded photo -> simplified human-drawable SVG -> ordered strokes -> teaching steps.
What does the creator edit?	Order/group/split/merge/replay/instructions - not manual path drawing.
What is the shared contract?	Existing tutorial.schema.json v1 + SVG parser semantics + conformance corpus.
Where does approved content go?	shared/Tutorials plus proposed shared catalog/reference metadata.
What is the first proof?	Five strong Houses lessons end-to-end.
What is the eventual learner outcome?	See a real subject, know how to begin, draw without immediate frustration, and enjoy filling a physical sketchbook.